THE TRADING DEVELOPMENT KIT

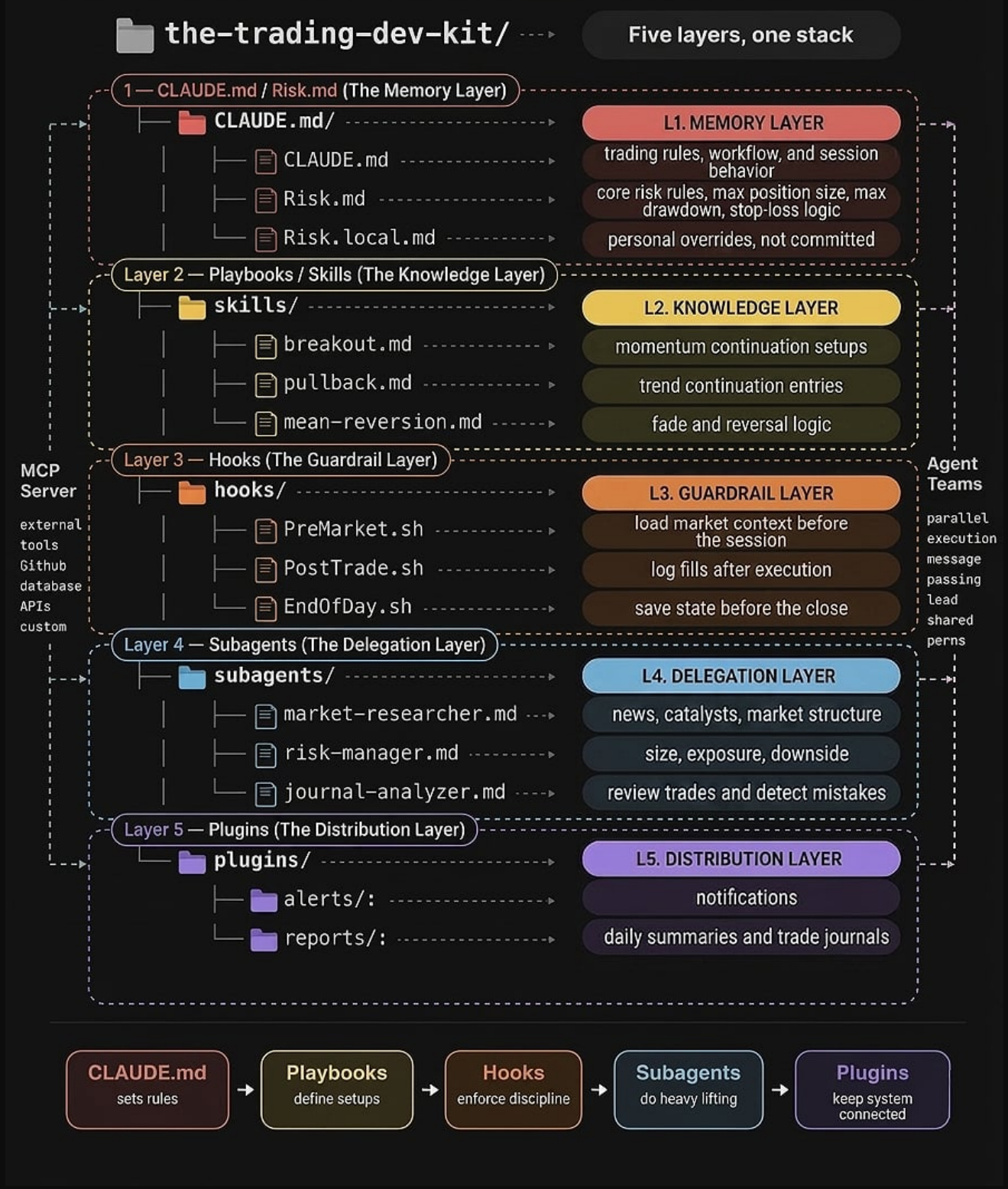
A Five-Layer System for Building a Consistent, Rule-Based Trading Operation with Claude

L1	**CLAUDE.md / Risk.md**	Memory Layer	Rules, risk, session behavior
L2	**Playbooks / Skills**	Knowledge Layer	Setup library and trade logic
L3	**Hooks**	Guardrail Layer	Automation around session events
L4	**Subagents**	Delegation Layer	Specialist agents for heavy work
L5	**Plugins**	Distribution Layer	Bundle, package, and ship your system

Write your system once. Stop donating to the market.

The Trading Development Kit

(CLAUDE.md + Risk + Playbooks + Hooks + Subagents + Plugins)



THE STACK AT A GLANCE

Overview: Five Layers, One System

The Trading Development Kit is built around a single principle: discretion loses, systems win. Every decision you leave to the moment is a decision you will get wrong under pressure. This kit structures your entire trading operation into five distinct layers, each with a clear job and a clear boundary.

HOW THE LAYERS CONNECT

The five layers form a chain. CLAUDE.md sets the rules. Playbooks define the setups. Hooks enforce discipline through automation. Subagents do the heavy analytical work without cluttering your main session. Plugins distribute the finished system to your team or across environments.

None of these layers replace your edge. They protect it. A good setup executed inconsistently is worse than no setup at all, because it gives you false data about what works. This kit removes inconsistency from the equation.

WHAT EACH LAYER DOES

L1 - Memory Layer	CLAUDE.md and Risk.md are always loaded. They hold your trading rules, risk parameters, and session behavior. If it is not written here, it does not exist in your system.
L2 - Knowledge Layer	Playbooks are on-demand setup libraries. Breakout, pullback, mean-reversion. Each one is a precise, reusable definition of a trade setup you actually take.
L3 - Guardrail Layer	Hooks are shell scripts that fire automatically around session events. PreMarket loads context. PostTrade logs fills. EndOfDay saves state. No manual steps required.
L4 - Delegation Layer	Subagents are specialist Claude instances. Market researcher, risk manager, journal analyzer. They run in their own context window and return one clean answer.
L5 - Distribution Layer	Plugins bundle your entire system into a deployable package. Build once, install on any machine or share with your team in one click.

STARTER PROMPTS: OVERVIEW

LOAD MY FULL TRADING SYSTEM

```
Load CLAUDE.md, Risk.md, and the breakout playbook. Confirm each rule is active and
summarize my current session constraints.
```

SYSTEM HEALTH CHECK

```
Review all five layers of my trading system. Identify any gaps, conflicts between rules, or missing components.
```

SESSION BRIEFING

```
Run PreMarket.sh, load the relevant playbook for today, and give me a one-paragraph briefing on what I should be looking for this session.
```

Layer 1 - CLAUDE.md / Risk.md

(Trading rules + Risk rules + Session behavior)

A. What it is

Always loaded. Always active.

This is the trader's memory layer.

Rules > feelings.

If it is not written here, it is not part of the system.

B. Where it lives

GLOBAL

`~/.claude/CLAUDE.md`

Loaded for every session

- Core trading style
- Default workflow
- Session behavior
- Non-negotiable habits

PROJECT

`.claude/CLAUDE.md`

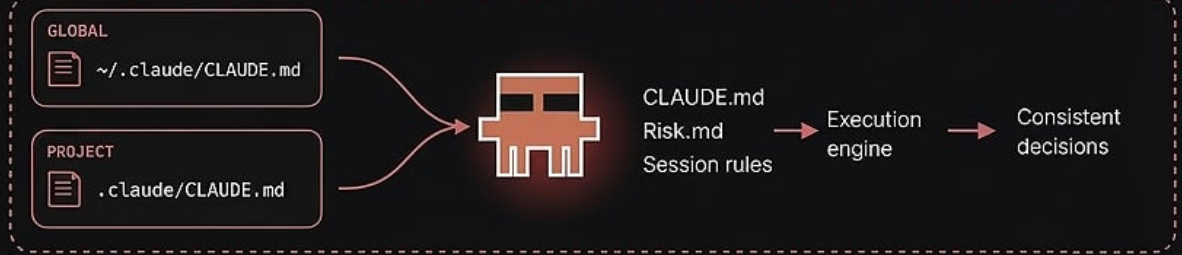
Loaded for this market / setup only

- Current playbock
- Market regime notes
- Setup-specific rules

C. What to put in it

- `trading.rules` → How you trade
  - Entry model
  - Confirmation rules
  - No-trade conditions
- `risk.rules` → How you survive
  - Max position size
  - Max daily drawdown
  - Stop-loss logic
- `execution.workflow` → How you execute
  - Pre-trade checklist
  - Entry trigger
  - Trade management
  - Exit plan
- `session.behavior` → How you behave
  - When you trade
  - When you stop
  - Tilt control
  - Review process

Write your system once.
Stop donating to the market.



LAYER 1

CLAUDE.md and Risk.md

The Memory Layer

Layer 1 is the foundation of the entire system. It is always loaded and always active. Every rule that governs how you trade, how you manage risk, and how you behave during a session lives here. The core principle is simple: rules over feelings. If a rule is not written in this layer, it is not part of your system.

TWO FILES, TWO JOBS

CLAUDE.md is your global memory. It loads for every single session regardless of what market or setup you are working on. It holds your core trading style, default workflow, session behavior, and non-negotiable habits. Think of it as the document that makes you the same trader every single day.

Risk.md lives alongside it and holds your core risk rules: maximum position size, maximum daily drawdown, and stop-loss logic. Risk.local.md handles personal overrides that are not committed to version control, giving you a private layer for account-specific adjustments.

WHAT TO PUT IN EACH SECTION

`trading.rules`	Entry model, confirmation rules, no-trade conditions. This defines how you get into a position and what circumstances will keep you out entirely.
`risk.rules`	Max position size, max daily drawdown, stop-loss logic. These are the numbers that keep you in the game tomorrow regardless of what happens today.
`execution.workflow`	Pre-trade checklist, entry trigger, trade management process, and exit plan. This is the sequence you follow every single time.
`session.behavior`	When you trade, when you stop, tilt control rules, and review process. This governs how you show up and when you walk away.

STARTER PROMPTS: LAYER 1

BUILD MY CLAUDE.MD

```
I trade [your style] on [timeframe]. My edge is [describe setup]. My max daily loss is [amount] and I never risk more than [%] per trade. Write my complete CLAUDE.md including trading.rules, risk.rules, execution.workflow, and session.behavior sections.
```

ENFORCE MY RULES MID-SESSION

```
Check my current position against Risk.md. Am I within my max position size and daily drawdown limits? Flag any violations immediately.
```

SESSION DEBRIEF

```
Review today's session against my CLAUDE.md rules. Which rules did I follow? Which did I break? Give me three specific corrections for tomorrow.
```

Layer 2 - Playbooks / Skills

(The Knowledge Layer)

A. What it is

On-demand. Auto-loaded.

This is the setup library.

The market changes. The playbook doesn't.

B. Where it lives

GLOBAL

`~/.claude/skills/`

Reusable across every market.

- Shared playbooks
- Reusable setup logic
- Trading patterns that work everywhere

PROJECT

`.claude/skills/`

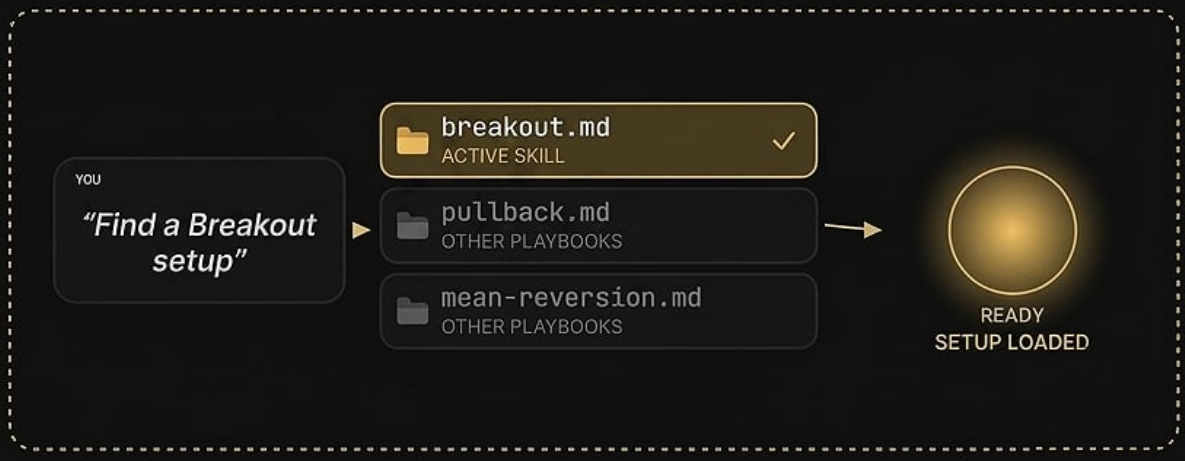
Built for this strategy only.

- Repo-specific edge
- Market-specific workflows
- Setup rules for this environment

C. What to put in it

- `breakout.md` → Momentum continuation setups
- `pullback.md` → Trend continuation entries
- `pullback.md` → Trend continuation entries
- `mean-reversion.md` → Fade and reversal logic

One setup. Wired forever.



LAYER 2

Playbooks and Skills

The Knowledge Layer

Layer 2 is your setup library. Playbooks are loaded on-demand, meaning Claude pulls in the relevant one based on what you are looking for in that session. Markets change in character constantly, but your playbook definitions should not. A breakout setup is a breakout setup. The precision lives in the file.

GLOBAL VS PROJECT PLAYBOOKS

Global playbooks live in ~/.claude/skills/ and are reusable across every market you trade. They hold your shared logic, the patterns and confirmation rules that apply regardless of instrument or environment.

Project playbooks live in .claude/skills/ inside a specific repository. These are built for one strategy or one market environment only. They hold repo-specific edge, market-specific workflows, and setup rules that only apply in that context.

THE THREE CORE PLAYBOOKS

`breakout.md`	Momentum continuation setups. Defines what constitutes a valid breakout, what confirmation is required, where the entry trigger sits, and how you manage the trade once it is active.
`pullback.md`	Trend continuation entries. Defines acceptable pullback depth, the structure needed to confirm trend continuation, and the criteria that invalidate the setup entirely.
`mean-reversion.md`	Fade and reversal logic. Defines extended conditions, the trigger for a fade entry, hard stop placement, and the profit target logic for mean-reversion trades.

STARTER PROMPTS: LAYER 2

WRITE A BREAKOUT PLAYBOOK

```
Write a breakout.md playbook for [your instrument]. Include: valid breakout structure, volume confirmation rules, entry trigger, initial stop placement, scaling logic, and the conditions that invalidate the setup.
```

LOAD AND APPLY A PLAYBOOK

```
Load pullback.md. Scan [instrument] on [timeframe] and identify any current setups that match the pullback criteria. For each one, specify entry, stop, and target.
```

COMPARE SETUPS AGAINST PLAYBOOK

```
I am looking at this chart: [describe or attach]. Score this setup against my breakout.md criteria from 1 to 10 and tell me what is missing before I can take the trade.
```

Layer 3 - Hooks

(The Guardrail Layer)

A What it is

Deterministic. Not discretionary.

Shell rules that fire around session events.

B How it triggers

MATCHER

```
$ PreMarket.sh
```

Runs before the session opens.

- Loads market context
- Reads overnight structure
- Sets bias, levels, and risk frame

COMMAND

```
$ PostTrade.sh
```

Runs after execution.

- Logs fills
- Saves trade metadata
- Captures what changed

C What hooks exist

PreMarket.sh	Load market context before the session
PostTrade.sh	Log fills after execution
EndOfDay.sh	Save state before the close

Hooks turn process into discipline.

(The Guardrail Layer)

1. Event fires
2. Matcher checks
3. Command runs

LAYER 3

Hooks

The Guardrail Layer

Hooks are deterministic. They do not ask for permission and they do not rely on memory. They are shell scripts that fire automatically around session events. The guardrail layer exists because discipline that depends on willpower will eventually fail. Discipline that is automated never does.

HOW HOOKS FIRE

Each hook follows a three-step sequence. First, an event fires: the session opens, a trade executes, the day ends. Second, a matcher checks whether the conditions are met. Third, the command runs automatically. There is no manual step in between.

THE THREE CORE HOOKS

`PreMarket.sh`	Runs before the session opens. Loads market context, reads overnight structure, and sets bias, key levels, and risk frame for the day. You start every session with the same information base.
`PostTrade.sh`	Runs after every execution. Logs the fill, saves trade metadata, and captures what changed in the position. Nothing falls through the cracks because nothing depends on you remembering to do it.
`EndOfDay.sh`	Runs before the close. Saves full session state, captures P and L data, and prepares the journal entry for review. The day is closed cleanly every time.

STARTER PROMPTS: LAYER 3

WRITE MY PREMARKET HOOK

```
Write a PreMarket.sh hook that: fetches overnight high and low for [instrument],
identifies the current market regime (trend or range), sets a bias for the session, and
outputs a structured pre-market briefing I can read in under 60 seconds.
```

POSTTRADE LOGGING

```
Write a PostTrade.sh hook that logs: instrument, direction, entry price, stop price,
size, fill time, and the setup type from my playbook. Save each entry to trades.log with a
timestamp.
```

END OF DAY REVIEW

```
Run EndOfDay.sh. Summarize my session: total trades, win rate, largest win, largest loss,
and whether I followed my CLAUDE.md rules. Flag any rule violations with specific
examples.
```

Layer 4 - Subagents

(The Delegation Layer)

A. What it is

Own context window.

Delegate work without cluttering the main session.

B. How it works

PARENT

main session

Where you run the book.

- Sets the objective
- Delegates the work
- Only sees the result

CHILD

subagent run

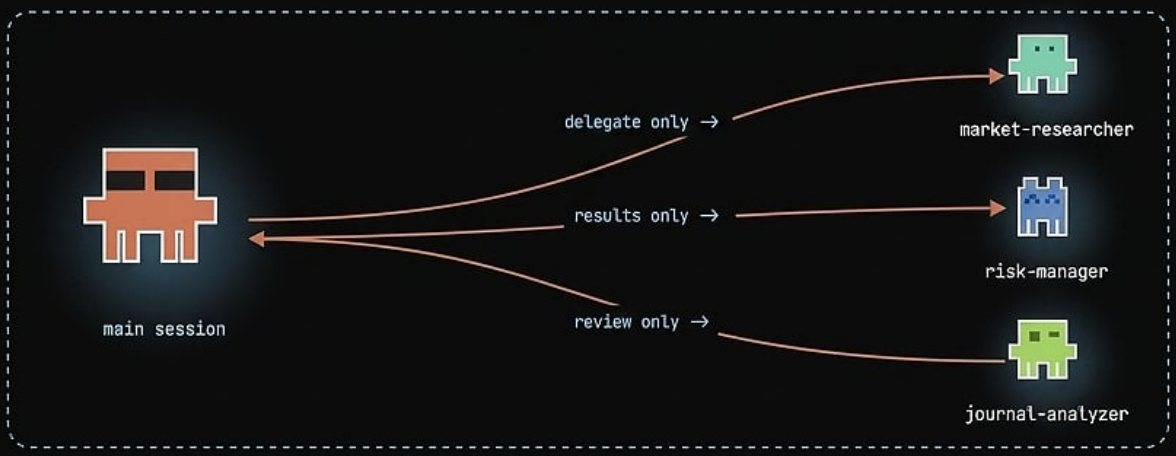
Spawned to do one job.

- Own prompt
- Own tools
- Own context window
- Returns one clean answer back

C. What subagents exist

market-researcher.md	News, catalysts, market structure
risk-manager.md	Size, exposure, downside
journal-analyzer.md	Review trades and detect mistakes

Delegate the noise. Keep the main thread clean.



LAYER 4

Subagents

The Delegation Layer

Subagents are specialist Claude instances that run in their own context window. The main session sets the objective and delegates the work. The subagent does the job and returns one clean answer. The main thread never sees the noise, only the result. This keeps your primary session focused and fast.

PARENT AND CHILD

The main session is the parent. It runs the book, sets objectives, and delegates tasks to specialist subagents. Each subagent is a child: spawned to do one job, with its own prompt, its own tools, and its own context window. When the job is done, it returns one clean answer and closes.

THE THREE CORE SUBAGENTS

`market-researcher.md`	Handles news, catalysts, and market structure analysis. You delegate the research task and receive a structured summary: what is happening, why it matters, and how it affects your bias for the session.
`risk-manager.md`	Handles size calculation, exposure analysis, and downside scenario modeling. You delegate the numbers and receive a position sizing recommendation with downside spelled out clearly.
`journal-analyzer.md`	Reviews your trade history and detects mistakes. You delegate the journal and receive a pattern analysis: what errors are repeating, what setups are performing, and what needs to change.

STARTER PROMPTS: LAYER 4

DEPLOY MARKET RESEARCHER

```
Spawn market-researcher subagent. Task: research [instrument] for today's session.
Return: top 3 catalysts, current market structure, any scheduled events in the next 4
hours, and a one-line session bias.
```

DEPLOY RISK MANAGER

```
Spawn risk-manager subagent. Account size: [amount]. Planned trade: [instrument],
direction [long/short], entry [price], stop [price]. Return: recommended size, dollar
risk, max adverse excursion scenario, and whether this trade fits within today's
remaining risk budget.
```

DEPLOY JOURNAL ANALYZER

```
Spawn journal-analyzer subagent. Review my last 30 trades in trades.log. Identify my top 3 recurring mistakes, my best performing setup type, and the one rule I break most often. Format as a structured report.
```

Layer 5 - Plugins

(The Distribution Layer)

A. What it is

Bundle. Package. Ship.

Turn your trading system into something deployable.

B. What's in a plugin

MANIFEST

plugin.json

Your system blueprint.

- What strategies are included
- What rules are active
- What triggers fire
- Versioned. Locked. No guesswork.

STORE

marketplace.url

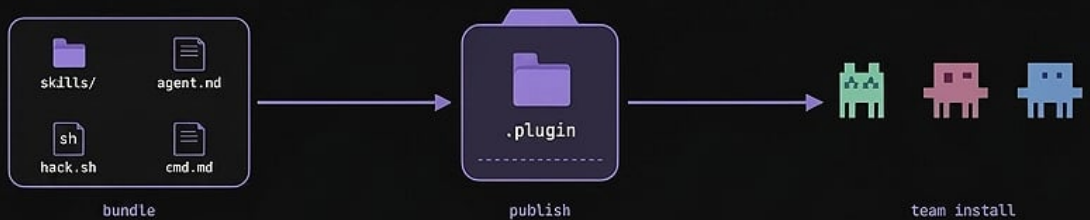
my-plugin
v1.0.0 · team-ready
Install

- Team can discover it
- One-click install
- Updates hit everyone instantly

C. What you can ship

skills/	The actual edge.
agents/	Entries, exits, size, hedge, risk handling.
hooks/	Alerts, conditions, market events.
commands/	Enter, scale, cut, hedge, close.

Build once. Install everywhere.



LAYER 5

Plugins

The Distribution Layer

Layer 5 turns your trading system into something you can ship. A plugin bundles your skills, agents, hooks, and commands into a single deployable package. Build it once. Install it on any machine. Share it with your team in one click. Every member of the team runs the same version of the same system.

WHAT A PLUGIN CONTAINS

Every plugin has two core components. The manifest is plugin.json: your system blueprint that lists what strategies are included, what rules are active, what triggers fire, and what version is running. The store is the marketplace URL where teammates can discover, install, and update your plugin instantly.

WHAT YOU CAN SHIP

`skills/`	The actual edge. Your playbooks, setup definitions, and confirmation logic packaged for any environment.
`agents/`	Entries, exits, sizing, hedging, and risk handling. The specialist subagents your system depends on, bundled and versioned.
`hooks/`	Alerts, conditions, and market event triggers. All the automation that runs around your session, ready to install.
`commands/`	Enter, scale, cut, hedge, close. The action layer that executes your decisions, standardized across the team.

STARTER PROMPTS: LAYER 5

BUILD A PLUGIN MANIFEST

```
Create a plugin.json manifest for my trading system. Include: system name, version 1.0.0, list of active strategies from my skills/ folder, list of agents in my agents/ folder, hooks that fire automatically, and the commands available to the team.
```

PACKAGE FOR TEAM DISTRIBUTION

```
Bundle my current system for team distribution. Include skills/, agents/, hooks/, and commands/. Write a README that explains how to install, what each component does, and what the team needs to configure before their first session.
```

AUDIT PLUGIN BEFORE SHIPPING

```
Review my plugin before I publish it. Check for: missing risk rules, hooks that reference non-existent files, agent prompts that conflict with CLAUDE.md, and any commands that could bypass Risk.md limits.
```

Build once. Install everywhere.

The Trading Development Kit is complete when all five layers work together as one system. Rules set in Layer 1. Setups defined in Layer 2. Discipline enforced in Layer 3. Analysis delegated in Layer 4. System shipped in Layer 5.